Experiment No: 6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- Automation: Executes actions automatically when conditions are satisfied.
- Transparency: The contract code and transactions are visible on the blockchain.
- Security: Once deployed, contracts are immutable and tamper-proof.
- Cost-effective: Removes intermediaries and reduces transaction costs.
- Trustless System: Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum

Implementation:

The project "Supply Chain Tracking System" focuses on tracking product movement and ownership transfer using Ethereum-based smart contracts.

We will implement two contracts:

1. SupplyChain.sol → Stores product details
2. ProductTransfer.sol → Handles transfer of ownership

CODE :

Supplychain.sol

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract SupplyChain {

    struct Product {

        uint id;

        string name;

        string location;

        address owner;

        bool isRegistered;

    }

    mapping(uint => Product) public products;

    address public admin;

    constructor() {

        admin = msg.sender;

    }

    function registerProduct(uint _id, string memory _name, string memory _location,
address _owner) public {

        require(msg.sender == admin, "Only admin can register product");

        require(!products[_id].isRegistered, "Product already registered");

        products[_id] = Product(_id, _name, _location, _owner, true);

    }
```

```
    function updateOwner(uint _id, address _newOwner) public {
        products[_id].owner = _newOwner;
    }
    function getProduct(uint _id) public view returns (uint, string memory, string memory,
address) {
        Product memory p = products[_id];
        return (p.id, p.name, p.location, p.owner);
    }
}
```

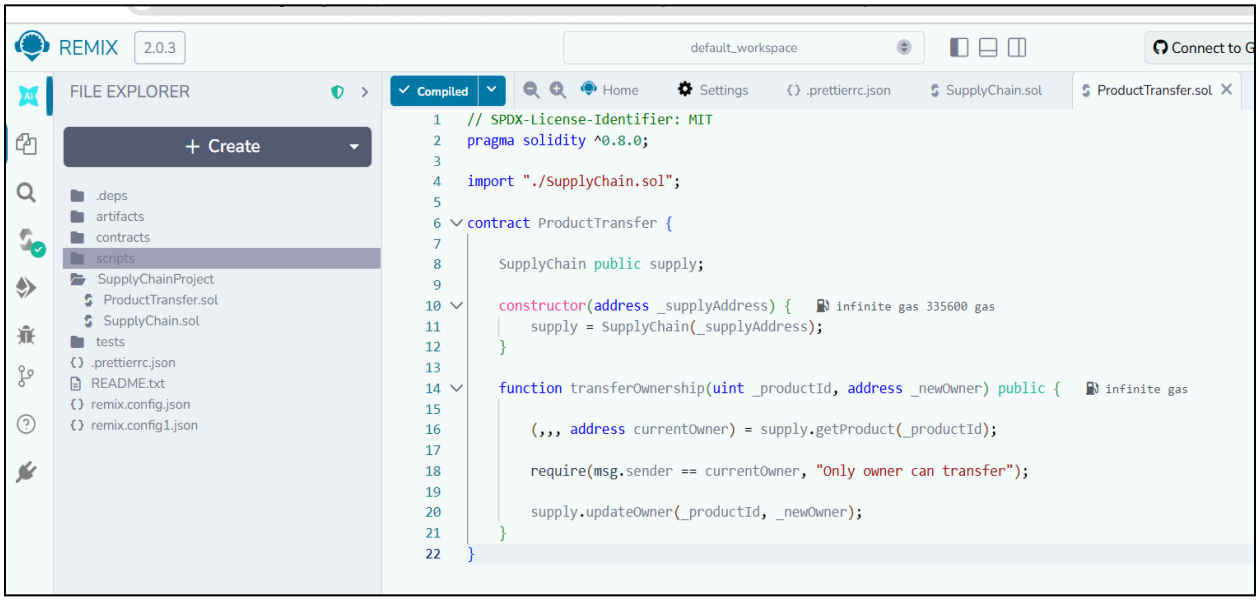
2.ProductTransfer.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
import "./SupplyChain.sol";
contract ProductTransfer {
    SupplyChain public supply;
    constructor(address _supplyAddress) {
        supply = SupplyChain(_supplyAddress);
    }
    function transferOwnership(uint _productId, address _newOwner) public {
        (,,, address currentOwner) = supply.getProduct(_productId);
     require(msg.sender == currentOwner, "Only owner can transfer");
```

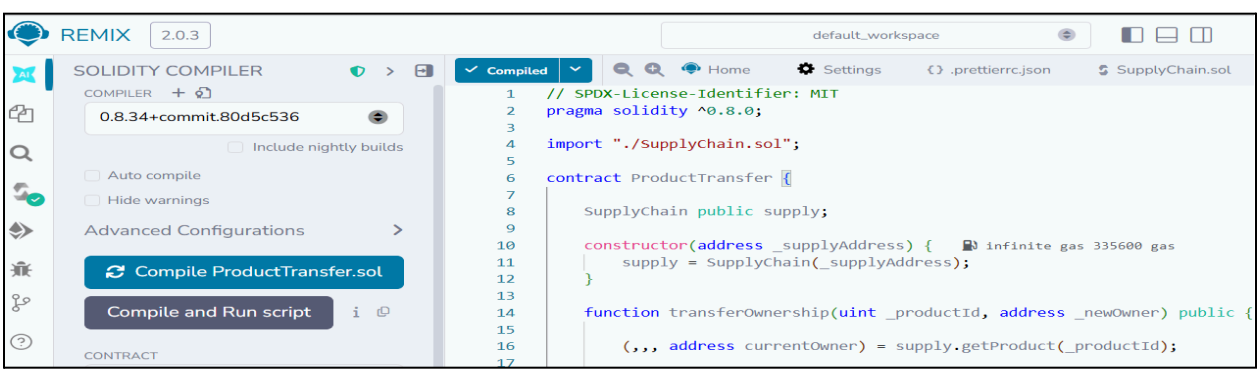
```
        supply.updateOwner(_productId, _newOwner);
    }
}
```

Step 1: Setup

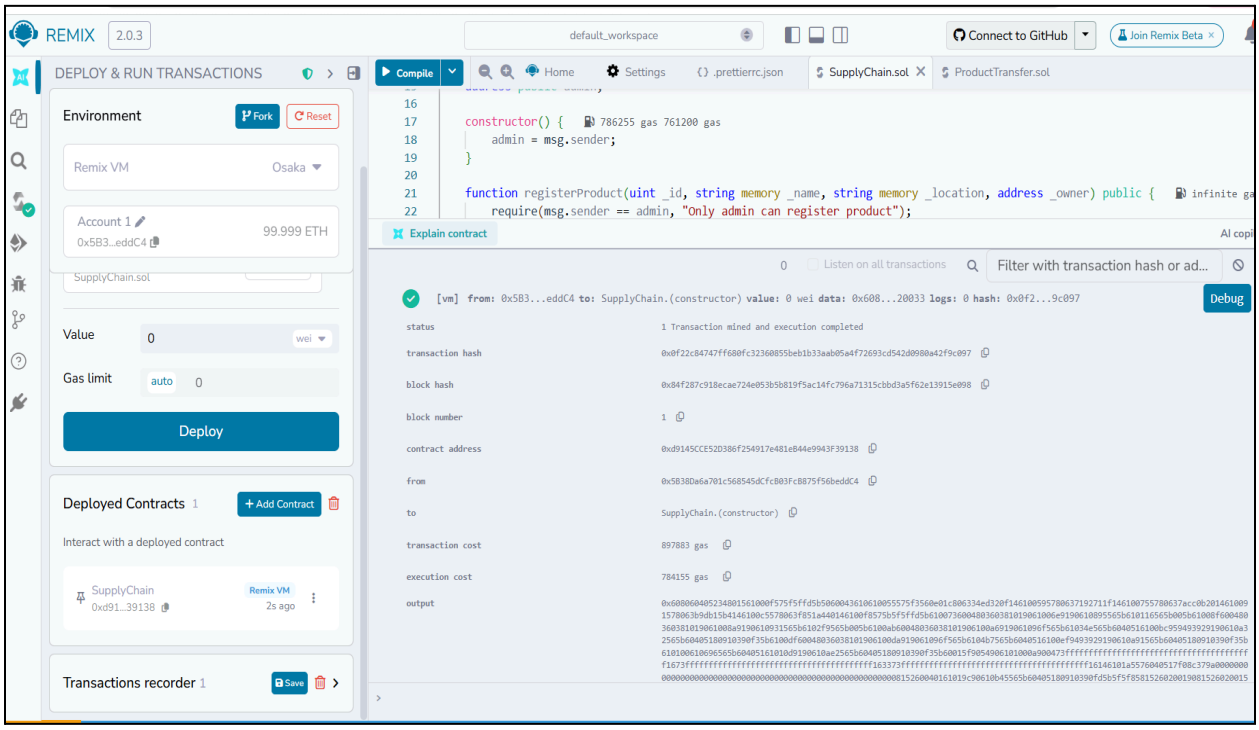
- Open Remix IDE
- Create folder: `SupplyChainProject`
- Create:
    - SupplyChain.sol
    - ProductTransfer.sol



Step 2: Compile both contracts

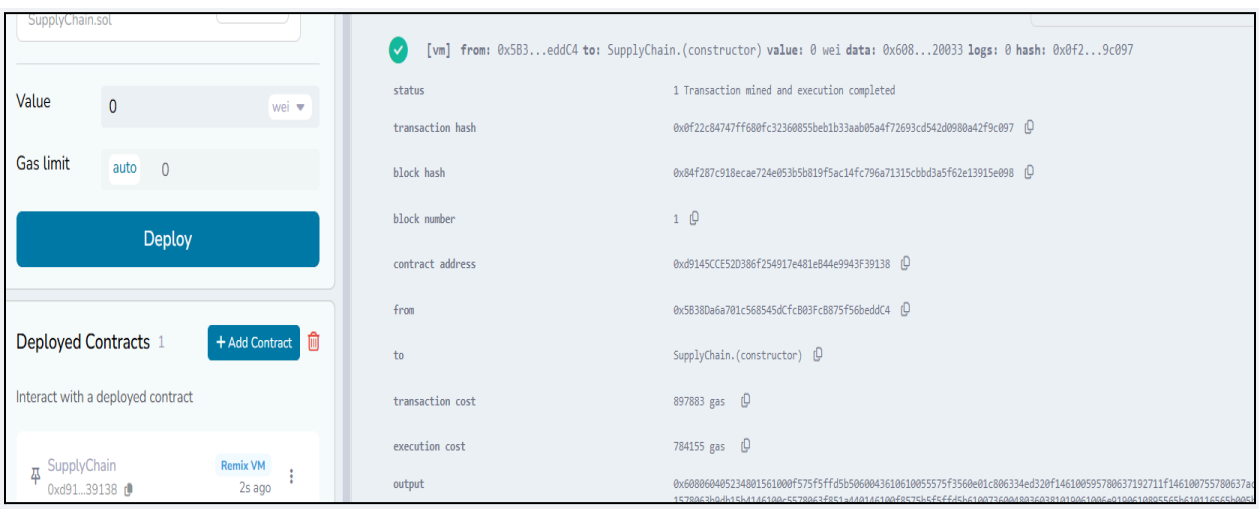


Step 3: Deploy SupplyChain contract

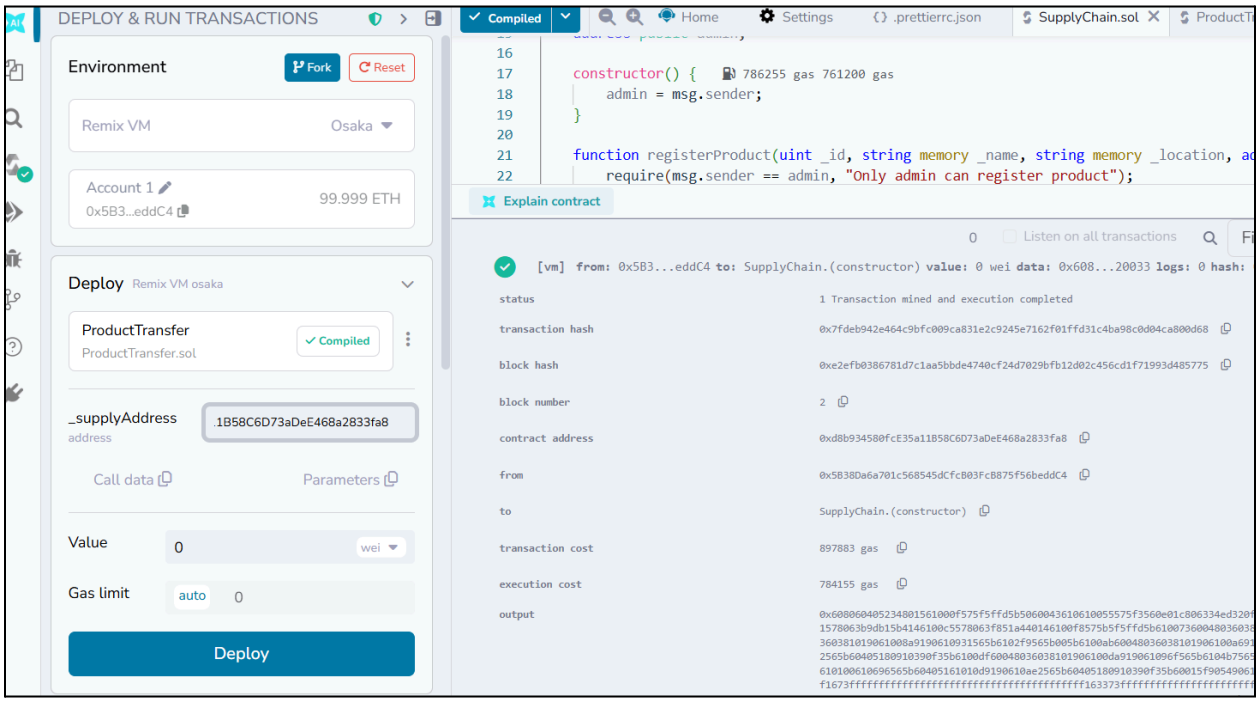


Step 4: Copy deployed contract address

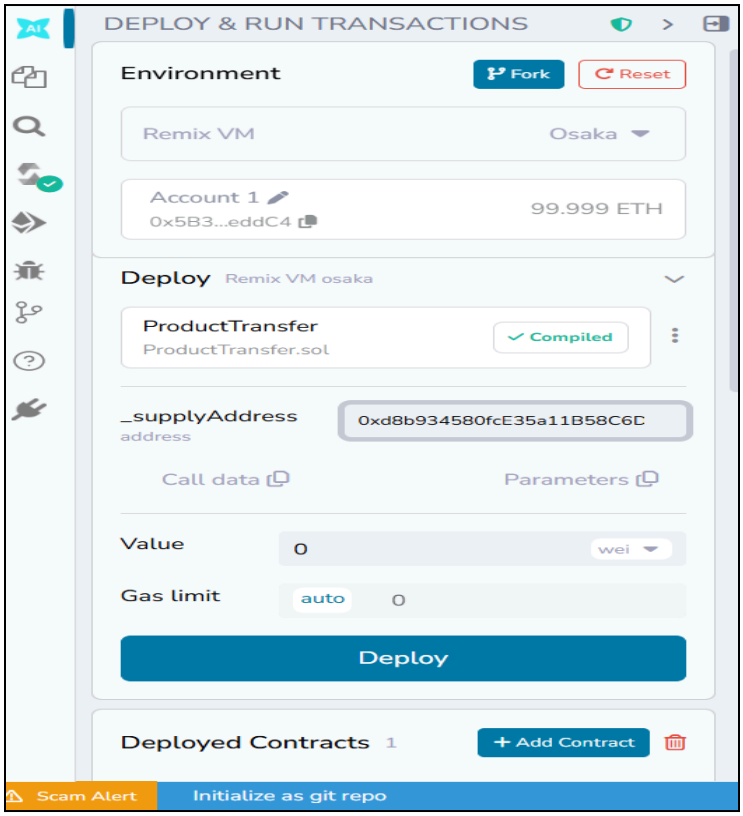
0xd9145CCE52D386f254917e481eB44e9943F39138



Step 5: Select ProductTransfer contract



Step 6: Paste address in `_supplyAddress` and deploy

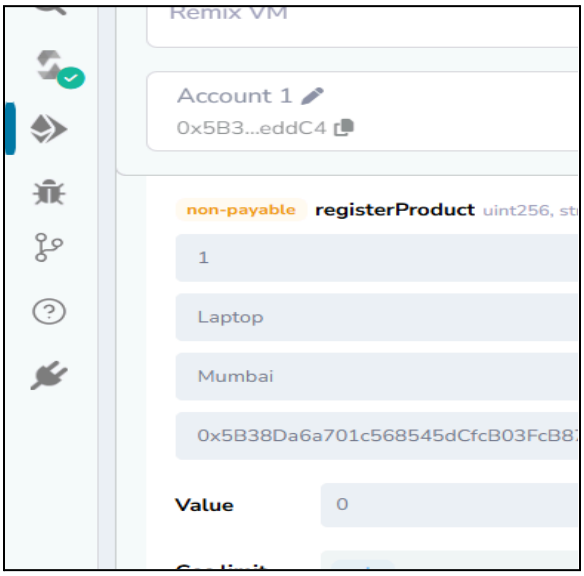


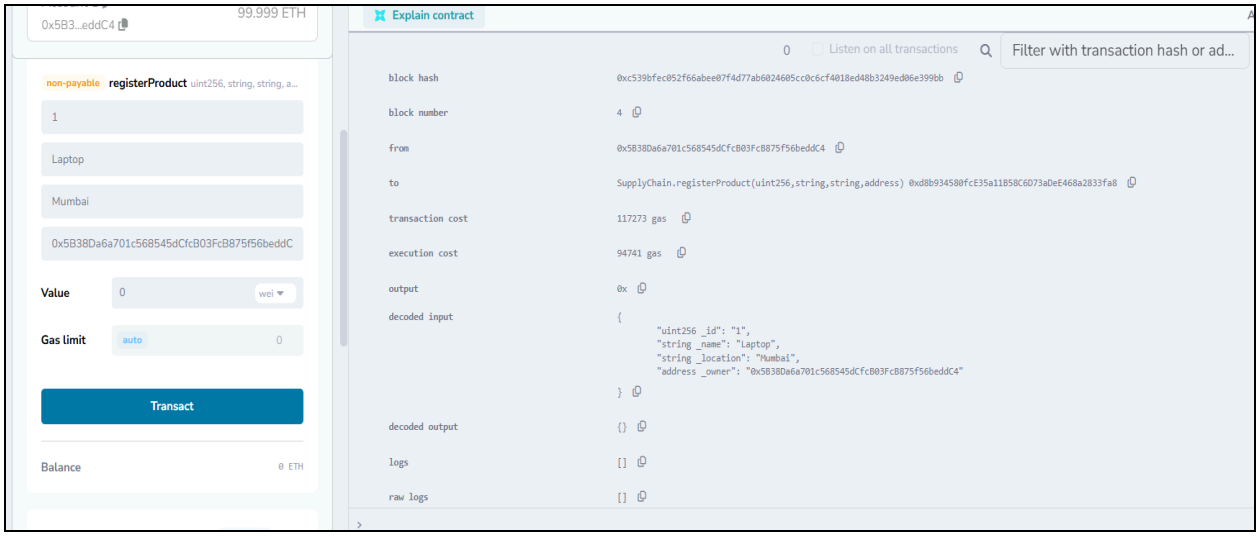
Step 7: Transactions

1. Register Product

- id = 1
- name = "Laptop"
- location = "Mumbai Warehouse"
- owner = your wallet

Click transact 👉 Check using getProduct(1)



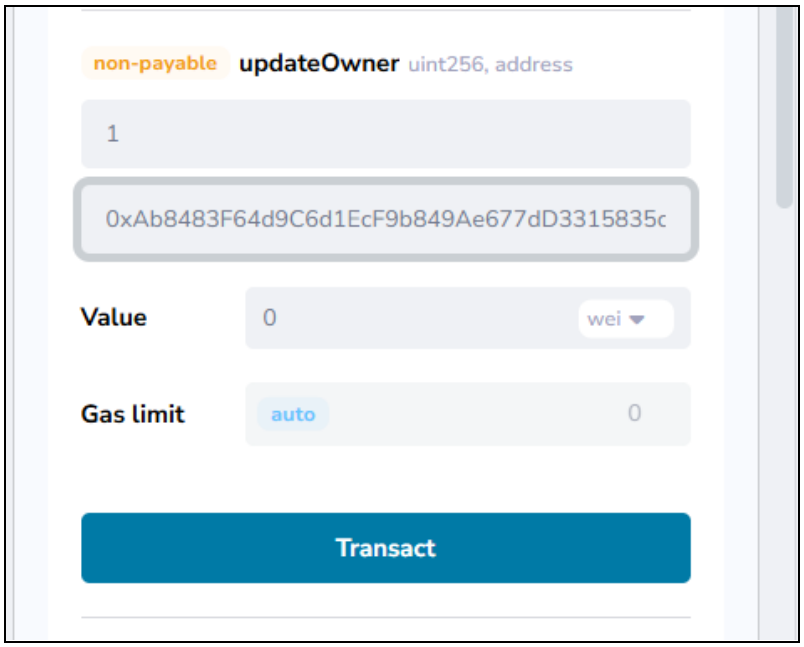


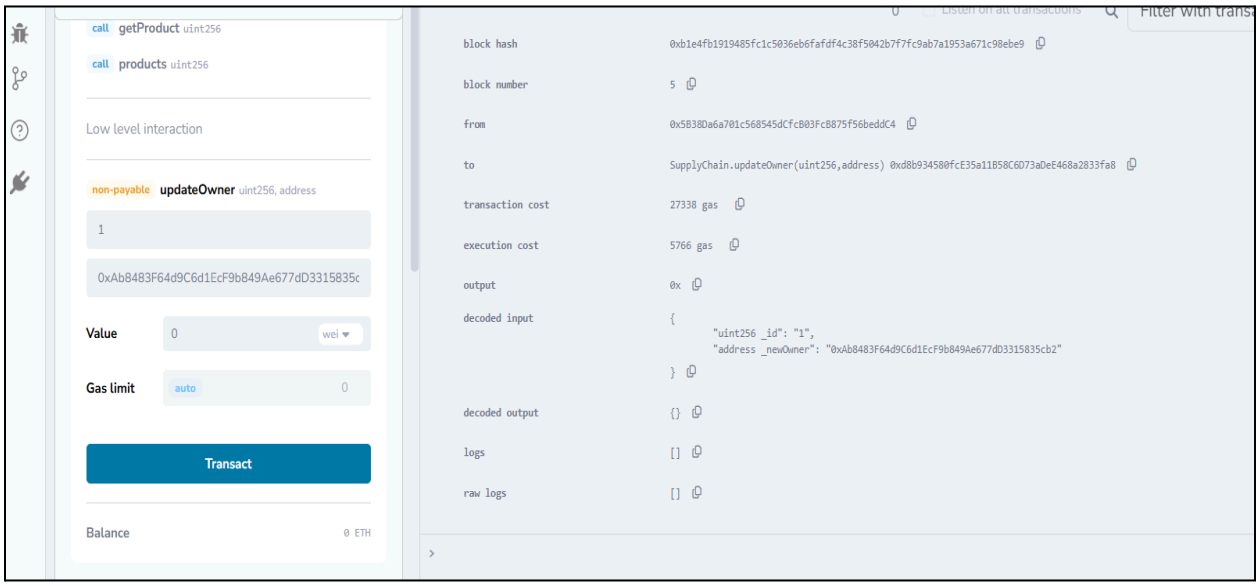
2. Transfer Ownership

- Copy another wallet address (buyer)
- Use:
  - _productId = 1
  - _newOwner = buyer address

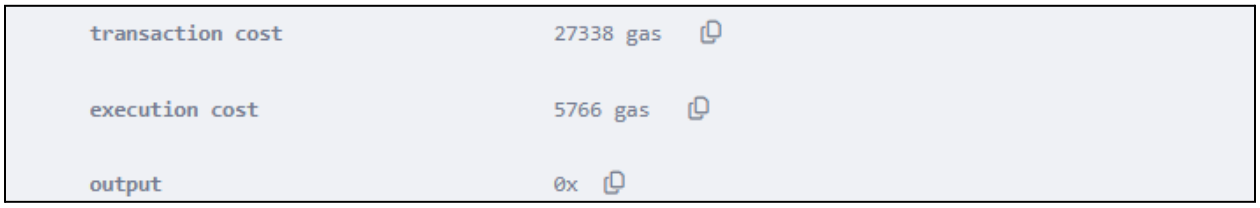
Click transact

👉 Verify using getProduct(1) → owner will change





Step 8. check gas



Conclusion:

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.